CL3 Viva Theory & Code Walkthrough

CL3 Viva Theory & Code Walkthrough

A single, exhaustive reference for the 10 practicals in **417534: Computer Laboratory III**. For each experiment you will find:

1. **Aim** — restated.
2. **Theory** — every concept, term, and acronym you should be able to explain.
3. **Why this language / framework?** — justification for the technology choice.
4. **Code walkthrough** — what each part of the code actually does and why.
5. **Likely viva questions with answers**.

A short *cross-cutting* Q&A section appears at the very end with questions that span multiple practicals.

---

0. Common Foundations

Examiners often start with broad questions before drilling into a specific practical. Be ready for these.

0.1 What is a distributed system?

A collection of independent computers (nodes) connected by a network that **appear to the user as a single coherent system**. They cooperate by passing messages over the network. Defining property: there is **no shared memory** and **no global clock** — synchronization happens only via message passing.

Key goals: **transparency** (users don't see the distribution), **scalability** (add machines to grow capacity), **fault tolerance** (failure of one node does not bring the whole system down), **resource sharing**, **concurrency**.

0.2 Concurrency vs Parallelism vs Distribution

- **Concurrency** — multiple tasks make progress in overlapping time periods on possibly one CPU (interleaved execution).
- **Parallelism** — multiple tasks execute literally at the same instant on multiple CPU cores.
- **Distribution** — tasks run on **different machines** connected by a network.

A distributed system is normally also concurrent, often parallel, but the converse is not true.

0.3 Inter-process communication (IPC)

Mechanisms to allow processes to exchange data: - **Sockets** (raw TCP/UDP) - **Remote Procedure Call (RPC)** — call a function on another machine - **Remote Method Invocation (RMI)** — like RPC but call methods on remote *objects* - **Message queues /**

brokers (RabbitMQ, Kafka) - **Shared memory** (only on the same machine) - **Pipes / named pipes** - **REST / gRPC / GraphQL APIs over HTTP**

0.4 Synchronous vs Asynchronous

- **Synchronous** — caller blocks until the remote operation completes (RPC, RMI).
- **Asynchronous** — caller continues immediately; the result arrives later via a callback, future/promise, or event (message queues, WebSockets, async/await).

0.5 Stateful vs Stateless

- **Stateful** — server remembers information between calls (e.g., the Hotel Booking server keeps the bookings map).
- **Stateless** — every request carries everything it needs (REST APIs are typically stateless). Stateless services are easier to scale and load-balance.

0.6 Marshalling / Serialization

Converting an in-memory object into a byte stream that can travel across a network. The reverse is **unmarshalling / deserialization**. RPC and RMI both rely on this.

0.7 Stub and Skeleton

- **Stub** (a.k.a. **proxy**): a client-side object that *looks like* the remote service. When the client calls a method on the stub, the stub serializes the arguments, sends them across the network, waits for the reply, and deserializes the result. From the caller's perspective the call looks local.
- **Skeleton**: the corresponding server-side object that receives the network message, deserializes the arguments, calls the real method, and sends the return value back.
- In modern Java RMI (Java 1.5+), the skeleton is generated dynamically — you no longer need `rmic` to compile one.

0.8 Failure semantics for remote calls

- **At-most-once** — call executes once or not at all (most desirable; used by Java RMI).
- **At-least-once** — may execute multiple times due to retries (idempotent operations only).
- **Exactly-once** — extremely hard in the presence of failures; usually approximated.
- **Maybe** — caller has no guarantee at all.

Practical 1 — RPC: Distributed Factorial

1.1 Aim

Build a distributed application using **Remote Procedure Call** in Python. The client sends an integer `n` to the server; the server returns `n!`.

1.2 Theory

1.2.1 What is RPC?

Remote Procedure Call is a paradigm where a program calls a procedure (function) that runs on another machine *as if it were a local procedure*. The transport details (network, packets, serialization) are hidden by the RPC framework.

Steps in any RPC call: 1. The client invokes a stub procedure locally. 2. The stub marshals (serializes) the call arguments into a network message. 3. The message is transmitted to the server (typically over TCP/IP). 4. The server skeleton receives the message, unmarshals the arguments. 5. The skeleton invokes the actual server-side procedure. 6. The return value goes back the same way (marshal → transmit → unmarshal → return).

1.2.2 History / examples of RPC frameworks

- **Sun RPC / ONC RPC** (1980s, used by NFS).
- **DCE RPC** — basis of Microsoft RPC.
- **XML-RPC** (1998), evolved into **SOAP**.
- **Pyro / Pyro4 / Pyro5** — Pythonic RPC.
- **gRPC** — modern, language-neutral, protobuf-based, used everywhere today.
- **Apache Thrift**, **Cap'n Proto**.

1.2.3 Pyro4 specifics

- **Pyro** = "Python Remote Objects".
- Each Pyro object is registered with a **Daemon** (a thread that listens on a socket).
- Registration returns a **URI** of the form `PYRO:<objectId>@<host>:<port>`. The URI is the network "address" the client uses to find the object.
- Pyro uses **pickle** (or `serpent`) for serialization. By default Pyro4 uses `serpent` for security reasons (pickle can execute arbitrary code).
- Optionally Pyro4 has a **Name Server** so clients can look up objects by friendly name; we don't use it here for simplicity.

1.2.4 Factorial

Mathematically, $n! = 1 \times 2 \times 3 \times \ldots \times n$, with $0! = 1$ by convention. Defined for non-negative integers. Grows extremely quickly ($20! \approx 2.4 \times 10^{18}$).

1.3 Why Python (Pyro4)?

- **Python** is concise; the demo can be written in ~10 lines per side, perfect for a lab exercise.
- **Pyro4** is a pure-Python RPC framework, so installation is `pip install Pyro4` and it works on any OS.
- Python integers are **arbitrary precision**, so factorial of large numbers does not overflow (Java would need `BigInteger`).
- Could we use Java RPC? Yes (Sun RPC, Apache XML-RPC, gRPC) — but Pyro4 is the simplest option that demonstrates the concept end-to-end.

1.4 Code walkthrough

`server.py`

```
import Pyro4
```

Imports the Pyro4 library.

```
@Pyro4.expose
class Factorial:
    def __init__(self):
        self.calls = 0
```

The `@Pyro4.expose` **decorator** declares which class (and which methods inside it) are remotely callable. Without it Pyro4 refuses to expose anything — a security default. `self.calls` keeps a per-instance counter.

```
def fact(self, n):
    self.calls += 1
    ...
    if n < 0: raise ValueError(...)
    f = 1
    for i in range(1, n + 1): f *= i
    return f
```

The remote method. We validate input ($n \geq 0$), then compute factorial iteratively. The `ValueError` is **automatically propagated back to the client** by Pyro4 — RPC frameworks generally serialize exceptions too.

```
daemon = Pyro4.Daemon()
uri = daemon.register(Factorial())
```

Creates a Pyro4 daemon (a TCP listener) and registers a `Factorial` instance with it. `uri` is a unique address like `PYRO:obj_xxx@localhost:39845`.

```
daemon.requestLoop()
```

Blocks forever, dispatching incoming RPC calls to the registered object. Equivalent to `while True: handle one call`.

`client.py`

```
proxy = Pyro4.Proxy(uri)
proxy._pyroBind()
```

`Pyro4.Proxy` builds a client-side **stub** for the remote object. `_pyroBind()` opens the network connection eagerly so we fail fast if the URI is wrong.

```
result = proxy.fact(n)
```

Looks like a local method call but actually serializes `n`, sends it to the server, waits for the reply, and returns the deserialized result.

`time.perf_counter()` around the call measures **round-trip time** so the user can see network latency.

1.5 Likely viva questions

Q: What is RPC? A: A protocol that lets a program invoke a procedure on a remote machine as if it were local. The framework hides network communication and serialization.

Q: Difference between RPC and a normal function call? A: A normal call has no marshalling, no network, executes in the same process, and is faster by orders of magnitude. RPC adds serialization, network round-trip, can fail with network errors, and the parameters must be serializable.

Q: What does `@Pyro4.expose` do? A: It marks a class/method as remotely accessible. Anything not exposed is invisible to clients, preventing accidental remote access to private logic.

Q: What is a URI in Pyro4? A: A string of the form `PYRO:<objectId>@<host>:<port>` that uniquely identifies a remote Pyro object. Clients use the URI to construct a Proxy.

Q: Synchronous or asynchronous? A: Pyro4 calls are synchronous by default — the client blocks until the result returns. Pyro4 also supports `oneway` calls (fire-and-forget) and async via futures.

Q: How are exceptions handled across the network? A: They are serialized on the server, transmitted, and re-raised on the client. Pyro4 even preserves the remote stack trace.

Q: What is marshalling/unmarshalling? A: Converting in-memory objects to a byte stream (marshal) and back (unmarshal). Required because networks transmit bytes, not

Python objects.

Q: Why does the client need the URI? A: Without a name server, the URI is the only way to find the object on the network. With a name server, the client could do `Pyro4.locateNS().lookup("factorial.service")`.

Practical 2 — Java RMI: String Concatenation

2.1 Aim

A distributed application using **Java RMI**. Client sends one or more strings; server returns concatenated/transformed result.

2.2 Theory

2.2.1 What is RMI?

Remote Method Invocation is Java's native RPC mechanism. Unlike generic RPC, RMI lets you call **methods on remote objects** and pass entire objects (including custom ones) as arguments. Built on top of TCP. Uses **Java Object Serialization** for marshalling.

2.2.2 RMI Architecture (3 layers)

1. **Stub / Skeleton layer** — proxies on each side that hide marshalling.
2. **Remote Reference layer** — handles object identity, parameter passing semantics, garbage collection.
3. **Transport layer** — TCP sockets, connection management.

2.2.3 Key classes / interfaces

- `java.rmi.Remote` — marker interface every remotable interface must extend.
- `java.rmi.RemoteException` — every remote method must declare it; thrown on network failure.
- `java.rmi.server.UnicastRemoteObject` — superclass that turns an object into a remote object (sets up sockets).
- `java.rmi.registry.Registry` / `LocateRegistry` — the **RMI registry**, a simple name service mapping names → remote object stubs.
- `java.rmi.Naming` — convenience API for binding/lookup with `rmi://host/name` URLs.

2.2.4 RMI Registry

A small daemon that holds `name → remote object stub` mappings. Default port **1099**. Started by: - `rmiregistry` command, **OR** - programmatically via `LocateRegistry.createRegistry(1099)` (what we use, because the external command sometimes needs the classpath set up correctly and is fragile).

2.2.5 bind vs rebind

- `bind(name, obj)` — fails if the name is already taken.
- `rebind(name, obj)` — replaces any existing entry. Usually preferred so you can restart the server without unbinding.

2.2.6 Pass-by-value vs pass-by-reference in RMI

- **Local objects** (Serializable but not Remote) are passed **by value** — a copy is sent over the wire.
- **Remote objects** (subclasses of `UnicastRemoteObject`) are passed **by reference** — only the stub travels; the original lives on the server.

2.2.7 Java Object Serialization

Java's built-in mechanism to convert objects → bytes. Class must implement `Serializable`. Strings, primitives, and most JDK collections are already serializable. Used internally by RMI.

2.3 Why Java for RMI?

- **RMI is built into the JDK** (`java.rmi.*`); no third-party libraries needed.
- **Java has cross-platform bytecode**, so the same class can run unchanged on client and server.
- **Object Serialization** is also built in, making method arguments/returns of arbitrary complexity easy to pass.
- Compared to Python: RMI gives stronger type checking at compile time (the `StringConcat` interface enforces method signatures).
- Compared to gRPC: no IDL/protobuf to write — the Java interface itself is the contract.

2.4 Code walkthrough

`StringConcat.java` (interface)

```
public interface StringConcat extends Remote {
    String concat(String a, String b) throws RemoteException;
    String concatAll(List<String> parts, String separator) throws RemoteException;
    ...
}
```

- `extends Remote` — makes this an RMI-callable interface (without it, RMI ignores the methods).
- Every method declares `throws RemoteException` — mandatory; thrown on network failure.
- The interface is what both client and server import. It is the **contract**.

`Server.java`

```
public class Server extends UnicastRemoteObject implements StringConcat {
    Server() throws RemoteException {}
```

- `extends UnicastRemoteObject` — gives the object a network identity. The constructor must declare `RemoteException` because the superclass does.

```
Registry registry = LocateRegistry.createRegistry(1099);
Server obj = new Server();
registry.rebind("concat", obj);
```

- Creates the registry on port 1099 (in this JVM — no separate `rmiregistry` process needed).
- Constructs a `Server` instance.
- Binds the instance under the name `"concat"`. The registry stores its **stub**.

When the client looks up `"concat"`, the registry returns the stub. The stub knows the host/port of the actual object and forwards method calls there.

`Client.java`

```
Registry registry = LocateRegistry.getRegistry("localhost", 1099);
StringConcat obj = (StringConcat) registry.lookup("concat");
```

- Connects to the registry; calls `lookup` to obtain the stub.
- The cast is to the *interface* (not the implementation class — the client never sees `Server.java`).

```
obj.concat("Hello", "World")
```

Looks like a local method call but goes through the stub: marshal → network → server → execute → marshal → network → unmarshal → return.

2.5 Likely viva questions

Q: What is RMI? A: Java's mechanism for invoking methods on objects that live in another JVM, possibly on another host. Built on serialization and TCP.

Q: Difference between RPC and RMI? A: RMI is **object-oriented** (calls methods on remote objects, passes objects), RPC is procedure-oriented (calls functions). RMI is Java-only; RPC is language-neutral. RMI uses Java serialization; classic RPC uses XDR (Sun RPC) or XML/JSON/protobuf.

Q: Why must remote interfaces extend `Remote`? A: It tells the RMI runtime that methods of this interface are remotely callable, and triggers stub generation. Without it, RMI treats the type as an ordinary local interface.

Q: Why does every remote method declare `RemoteException`? A: To force the caller to handle network failures (host unreachable, marshalling failure, etc.). It is a checked exception — compilation fails if you don't declare or catch it.

Q: What is `UnicastRemoteObject`? A: A superclass that, when extended, automatically exports the object — i.e., it opens a server socket, generates a stub, and registers the object with the RMI runtime. "Unicast" means it is reachable via a single TCP connection (vs. activatable / multicast).

Q: What is the RMI registry, and on what port does it run? A: A simple naming service that maps human-friendly names to remote object stubs. Default port **1099**.

Q: Difference between `bind` and `rebind`? A: `bind` fails if the name is already in use; `rebind` overwrites silently.

Q: Are arguments passed by value or by reference? A: Local (non-Remote) Serializable objects are passed by value (a copy is sent). Remote objects are passed by reference (only the stub travels).

Q: What is a stub? A: A client-side proxy implementing the same interface as the remote object. It handles marshalling, network transport, and unmarshalling, so callers see method calls as if local.

Q: How does RMI achieve `at-most-once` semantics? A: TCP guarantees ordered, reliable delivery; the JVM throws `RemoteException` if the call fails after the message has been sent, and does *not* automatically retry — preserving at-most-once semantics.

Practical 3 — MapReduce Word Count on Hadoop

3.1 Aim

Use the **Hadoop MapReduce** framework to count occurrences of each word in a text file across a (pseudo-)cluster.

3.2 Theory

3.2.1 The Big Data motivation

A single machine cannot store or process petabytes of log/clickstream/sensor data. Solution: spread data and computation across many commodity machines. Move **computation to the data** instead of pulling all data over the network.

3.2.2 Hadoop ecosystem

- **HDFS** (Hadoop Distributed File System) — stores files split into 128 MB blocks and replicated across nodes.
- **YARN** (Yet Another Resource Negotiator) — cluster resource manager since Hadoop 2.x.
- **MapReduce** — the original processing framework. Today often replaced by Spark, but still important conceptually.
- Higher-level tools: Hive (SQL on Hadoop), Pig, HBase, Sqoop, Oozie, etc.

3.2.3 HDFS architecture

- **NameNode** — the master. Stores metadata: filenames, directory tree, block locations. Single point of failure (mitigated by HA in production).
- **DataNode** — workers. Store the actual blocks. Send heartbeats to the NameNode every 3s.
- **SecondaryNameNode** — *not* a hot standby. It periodically merges the NameNode's edit log with the fsimage to keep startup time bounded. Despite the name, it does **not** automatically take over on failure.
- Replication factor (default 3) — each block kept on 3 DataNodes for durability.

3.2.4 YARN architecture

- **ResourceManager** (RM) — global scheduler. Allocates cluster resources.
- **NodeManager** (NM) — runs on every worker; manages containers on that node.

- **ApplicationMaster** (AM) — one per running application; negotiates resources with RM, coordinates tasks.
- **Container** — bundle of CPU/RAM allocated to a task.
- Default RM port for client connections: **8032**. Web UI: **8088**.

3.2.5 MapReduce model

A user's computation expressed as two functions: - `map(k1, v1) → list(k2, v2)` — applied to every input record independently, in parallel. - `reduce(k2, list(v2)) → list(v3)` — applied to every distinct key after grouping, also in parallel.

Between the two phases, Hadoop performs **shuffle and sort**: it groups all values for the same key and ships them to the reducer responsible for that key.

3.2.6 Word Count in MapReduce

- Map: for each word in a line, emit `(word, 1)`.
- Shuffle: collect all `(word, 1)`, group by word.
- Reduce: for each word, sum the 1s.
- Output: `(word, totalCount)`.

3.2.7 Important MapReduce concepts

- **Combiner** — an optional mini-reducer that runs on the *map side* to reduce data sent over the network. For Word Count it can sum counts before shuffle.
- **Partitioner** — decides which reducer a key goes to. Default: `hash(key) % numReducers`.
- **InputFormat** / **RecordReader** — how Hadoop turns input files into key/value pairs. `TextInputFormat` (default) gives `(byteOffset, line)`.
- **OutputFormat** — writes reducer output. Default `TextOutputFormat` produces `key\tvalue` lines.
- **Speculative execution** — slow tasks are duplicated; first to finish wins. Improves tail latency.
- **Data locality** — Hadoop tries to schedule a map task on the node holding the input block, avoiding network transfer.

3.2.8 Hadoop Writable types

Hadoop has its own serialization format (smaller, faster than Java serialization) implemented through the `Writable` and `WritableComparable` interfaces. Common types: - `IntWritable`, `LongWritable`, `Text` (≈ `String`), `BooleanWritable`, `NullWritable`.

We use these instead of Java primitives for Map/Reduce keys and values.

3.2.9 Pseudo-distributed vs Standalone vs Fully-distributed

- **Standalone (local)** — one JVM, no daemons. Useful for development.
- **Pseudo-distributed** — all daemons (NameNode, DataNode, RM, NM) on one machine. What we use.

- **Fully distributed** — production: many machines, multiple DataNodes/NodeManagers.

3.3 Why Java for MapReduce?

- **Hadoop is written in Java**, and the native MapReduce API is Java. Programs run inside the same JVM as the Hadoop libraries, so no IPC overhead.
- Hadoop streaming allows Python/Ruby/etc., but **incurs extra per-record IPC** (data is piped via stdin/stdout to the external script).
- Java's **type safety** (generic Writable types in Mapper/Reducer signatures) prevents many runtime errors.
- Mature tooling: Maven/Gradle build, Eclipse/IntelliJ debug.
- Why not Spark Scala? Modern shops do prefer Spark, but the syllabus targets MapReduce specifically.

3.4 Code walkthrough

`WordMapper.java`

```
public class WordMapper extends Mapper<Object, Text, Text, IntWritable> {
    public void map(Object key, Text value, Context context) ...
```

- Generic params: `<INPUT_KEY, INPUT_VALUE, OUTPUT_KEY, OUTPUT_VALUE>`.
- For `TextInputFormat`, input key is the byte offset in the file (we ignore it), input value is the line as `Text`.
- We emit `(word, 1)` so the reducer can sum.

```
StringTokenizer st = new StringTokenizer(value.toString());
while (st.hasMoreTokens()) {
    context.write(new Text(st.nextToken()), new IntWritable(1));
}
```

`StringTokenizer` splits on whitespace by default. `context.write` is how Mappers and Reducers emit output. `new Text(...)` and `new IntWritable(1)` wrap Java types into Hadoop's serializable types.

`WordReducer.java`

```
public class WordReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    public void reduce(Text key, Iterable<IntWritable> values, Context context) ...
```

- The framework guarantees that for each unique `key`, exactly one `reduce` call is made with **all** the values.
- We loop and sum, then emit `(word, total)`.

`WordCount.java` **(driver)**

```
Job job = Job.getInstance(conf, "word count");
job.setJarByClass(WordCount.class);
job.setMapperClass(WordMapper.class);
job.setReducerClass(WordReducer.class);
job.setOutputKeyClass(Text.class);
job.setOutputValueClass(IntWritable.class);
FileInputFormat.addInputPath(job, new Path(args[0]));
FileOutputFormat.setOutputPath(job, new Path(args[1]));
System.exit(job.waitForCompletion(true) ? 0 : 1);
```

The driver wires Mapper, Reducer, output types, input/output paths, then submits the job and waits.

Job submission flow

1. Driver calls `waitForCompletion(true)`.
2. The client uploads the JAR + input split metadata to HDFS.
3. RM receives the request, launches an ApplicationMaster.
4. AM requests Map containers; NMs launch them; each runs `WordMapper`.
5. After all maps finish, shuffle/sort moves data to reducers.
6. AM launches Reduce containers; each runs `WordReducer`.
7. Output is written to HDFS as `part-r-00000`.
8. AM tells RM the application is done.

3.5 Likely viva questions

Q: What problem does MapReduce solve? A: Processing huge datasets that don't fit on a single machine, in parallel across a cluster, by expressing the computation as two simple functions (map, reduce) that the framework can automatically distribute, retry, and load-balance.

Q: What are the steps in MapReduce? A: Input split → Map → Shuffle and Sort → Reduce → Output.

Q: What does the Mapper output? A: Intermediate key/value pairs that the framework groups by key before sending to the reducer.

Q: What is the Combiner? A: A *local* reducer that runs on the mapper's machine to compress map output before shuffle, reducing network traffic. Must be associative and commutative for correctness.

Q: What is shuffle and sort? A: The phase between Map and Reduce: the framework partitions, sorts, and ships all values for the same key to the reducer assigned to that key.

Q: What is HDFS and why is its block size big (128 MB)? A: A distributed file system that stores files as large blocks across many DataNodes, replicated for fault tolerance. Big blocks minimise NameNode metadata and maximise sequential read throughput, matching the workload of analytics.

Q: Difference between SecondaryNameNode and a hot-standby NameNode? A: SecondaryNameNode does **not** take over on failure — it only periodically checkpoints the

NameNode’s metadata. A real HA setup uses a Standby NameNode plus ZooKeeper / JournalNodes.

Q: Why are we using Hadoop’s Writable types and not Java’s int/String? A: Writables have a custom binary format much smaller and faster than Java’s default serialization, and they are explicitly designed for Hadoop’s framework.

Q: What is YARN’s role? A: It is a cluster resource manager — abstracts CPU/RAM into containers and lets multiple frameworks (MapReduce, Spark, Tez) share the cluster.

Q: What ports do we need? A: 9000 (NameNode RPC), 9870 (NameNode UI), 8032 (RM), 8088 (RM UI).

Q: What is data locality and why does it matter? A: Hadoop schedules map tasks on the node hosting the input block. This avoids reading the block over the network — usually the slowest part of the job.

Practical 4 — Fuzzy Set Operations

4.1 Aim

Implement **union, intersection, complement, and difference** on fuzzy sets.

4.2 Theory

4.2.1 Crisp set vs Fuzzy set

- **Crisp set** — element either belongs (1) or does not (0). The world of classical (Boolean) logic.
- **Fuzzy set** — element belongs **to a degree** in [0, 1]. Each element x has a **membership value** μ(x). Captures vagueness: e.g., “tall” is not a yes/no property.

4.2.2 Membership function μ(x)

A function from the universe of discourse U → [0, 1]. Common shapes: - **Triangular** (linear up then down) - **Trapezoidal** - **Gaussian** - **Sigmoid**

4.2.3 Standard operations

For fuzzy sets A and B over the same universe:

Operation	Definition
Union (A ∪ B)	μ_A∪B(x) = max(μ_A(x), μ_B(x))
Intersection (A ∩ B)	μ_A∩B(x) = min(μ_A(x), μ_B(x))
Complement A’	μ_A’(x) = 1 - μ_A(x)
Difference A − B	μ_A−B(x) = min(μ_A(x), 1 − μ_B(x))
Cartesian product A × B	min(μ_A(x), μ_B(y))

These are the Zadeh / standard operators. Other choices exist (probabilistic operators, t-norms / t-conorms).

4.2.4 Properties

- **Idempotency**: A ∪ A = A, A ∩ A = A.
- **Commutativity**: A ∪ B = B ∪ A, A ∩ B = B ∩ A.
- **Associativity**: (A ∪ B) ∪ C = A ∪ (B ∪ C).
- **Distributivity**: A ∩ (B ∪ C) = (A ∩ B) ∪ (A ∩ C).
- **De Morgan’s laws**: (A ∪ B)’ = A’ ∩ B’, (A ∩ B)’ = A’ ∪ B’.

- **Excluded middle does NOT hold** in fuzzy logic — unlike Boolean: A ∪ A’ ≠ universe in general; A ∩ A’ ≠ ∅.

4.2.5 Applications of fuzzy sets

- Control systems (washing machines, ABS brakes, air conditioners adjust based on fuzzy rules).
- Decision making, AI, pattern recognition.
- Medical diagnosis (uncertain symptoms).
- Autopilot, robotics.

4.3 Why Python?

- Concepts dominate; fuzzy operations are simple list comprehensions — no need for performance optimisations.
- Python’s `max` / `min` and list comprehensions express the operations almost mathematically.
- Could be done in any language; Python is shortest and clearest for teaching.

4.4 Code walkthrough

```
def fuzzy_union(a, b):
    return [max(x, y) for x, y in zip(a, b)]
```

Element-wise `max` — exactly the Zadeh union definition.

```
def fuzzy_complement(a):
    return [round(1 - x, 4) for x in a]
```

`round(..., 4)` cleans up floating-point noise like `0.30000000000000004`.

The interactive part repeatedly reads sets, validates that values are in [0, 1], and lets the user pick an operation from a menu.

4.5 Likely viva questions

Q: What is a fuzzy set? A: A set where each element has a membership degree in [0, 1] rather than the binary in/out of a classical set.

Q: What is the membership function? A: A function μ : U → [0, 1] giving the membership degree of each element.

Q: Define fuzzy union/intersection. A: max / min of corresponding membership values.

Q: Why is the law of excluded middle violated? A: Because μ_A(x) and μ_A’(x) = 1 − μ_A(x) are both in [0, 1], so μ_A∪A’(x) = max(μ_A, 1−μ_A) which equals 1 only when μ_A is 0 or 1. For μ_A=0.5, the max is 0.5 — not 1.

Q: Give a real-world example of fuzzy logic. A: Air conditioner: rules like “IF temperature is HIGH AND humidity is HIGH THEN cooling is FAST”. The membership of “HIGH” is fuzzy, allowing smooth control rather than abrupt switching.

Q: Difference between probability and fuzzy membership? A: Probability is about the **likelihood of an event** in a yes/no world (0.7 chance of rain → either it rains or doesn’t). Fuzzy membership is about the **degree** to which something belongs to a category (a 35°C day has 0.7 membership in “hot”). Different mathematics, different interpretation.

Practical 5 — Load Balancing Simulation

5.1 Aim

Simulate distribution of incoming client requests across multiple servers using load-balancing algorithms.

5.2 Theory

5.2.1 What is a load balancer?

A component that **sits in front of a pool of servers and decides which server should handle each incoming request**, with the goals of: - Even resource utilisation - Higher throughput - Lower latency - High availability (route around failed servers) - Scalability (add servers dynamically)

5.2.2 Where it sits

Internet → DNS → Load Balancer → Pool of identical web/app servers → Database.

5.2.3 Layer 4 vs Layer 7

- **Layer 4 (transport)** balancing — uses TCP/UDP info (IP + port). Fast, simple, no inspection of application data.
- **Layer 7 (application)** — inspects HTTP headers, URLs, cookies. Allows path-based routing, sticky sessions, SSL termination.

5.2.4 Common algorithms

Algorithm	Description
Round Robin	Cycle servers in order: 0,1,2,0,1,2… Simple, fair if servers are identical.
Weighted Round Robin	Each server has a weight; higher weight → more turns. Good when servers have different capacities.
Random	Pick a random server uniformly. Statistically even for many requests.
Least Connections	Route to the server with the fewest active connections. Good for long-lived connections.
Least Response Time	Route to the fastest-responding server. Adapts to load.
IP Hash / Source IP	Hash the client IP to pick a server. Provides "stickiness" without cookies.
Consistent Hashing	Common in caching tiers (memcached); minimises remappings when servers come/go.

5.2.5 Health checks

The LB periodically pings each server (TCP open, HTTP 200 on `/health`). Unhealthy servers are removed from rotation until they recover.

5.2.6 Sticky sessions / session affinity

Once a client is routed to server X, subsequent requests from the same client also go to X. Achieved via cookies or IP hashing. Trade-off: simpler app code (in-memory session), but reduces load distribution.

5.2.7 Real-world load balancers

Hardware: F5, Citrix Netscaler. Software: **Nginx**, **HAProxy**, **Envoy**, **Apache Traffic Server**. Cloud: AWS ELB/ALB/NLB, GCP LB, Azure LB.

5.3 Why Python?

- Simulating an algorithm only requires a list, modulo arithmetic, and a print statement.
- No real network is involved — Python's expressiveness shines for showing the **logic** of each algorithm.
- For a real load balancer you'd use Nginx/HAProxy, not write one in Python.

5.4 Code walkthrough

```python
def round_robin(servers, n):
    return [servers[i % len(servers)] for i in range(n)]
```

The classic round robin: `i % len(servers)` cycles through indices 0, 1, …, N-1, 0, 1, …

```
def least_connections(servers, n):
    counts = {s: 0 for s in servers}
    assignments = []
    for _ in range(n):
        target = min(counts, key=counts.get)
        counts[target] += 1
        assignments.append(target)
    return assignments
```

Tracks per-server counts; each new request goes to the server with the smallest count.

The bar-chart distribution at the end gives a visual sense of fairness.

5.5 Likely viva questions

Q: Why do we need load balancing? A: To prevent any single server from being overloaded, to scale horizontally, to provide failover, and to give the appearance of a single endpoint to clients.

Q: Compare Round Robin and Least Connections. A: RR is stateless (no per-server tracking) and fair under uniform load. Least Connections requires tracking active connections but adapts to non-uniform request durations (good for long-lived connections like WebSockets, DB pools).

Q: Layer 4 vs Layer 7? A: L4 inspects only IP/port — fast, opaque to content. L7 inspects HTTP headers/URLs — enables routing rules, SSL termination, but slower.

Q: Sticky sessions advantages/disadvantages? A: Pro: simpler in-memory session state on app servers. Con: uneven distribution, hard to scale, broken when a server dies.

Q: What's a single point of failure? A: The load balancer itself! Mitigated by deploying multiple LBs behind DNS round-robin or using floating virtual IPs (keepalived / VRRP).

Q: How does AWS ELB do health checks? A: Periodic TCP/HTTP checks to a configured endpoint; unhealthy targets are removed from the pool.

Practical 6 — Clonal Selection Algorithm

6.1 Aim

Implement the **Clonal Selection Algorithm (CSA)**, an immune-system-inspired optimization technique.

6.2 Theory

6.2.1 Biological inspiration

The vertebrate adaptive immune system fights pathogens (antigens) by: 1. **B-cells** carrying receptors (antibodies) circulate. 2. When a B-cell's receptor matches an antigen with high affinity, the cell is **selected**. 3. The cell **proliferates (clones)** itself rapidly. 4. Clones undergo **somatic hypermutation** — small random changes — increasing affinity. 5. Best-matching clones become memory cells; future infections are eliminated quickly.

6.2.2 Clonal Selection Algorithm (CLONALG)

Maps these biology steps onto computer optimization: 1. Random population of antibodies (candidate solutions). 2. Compute **affinity** (fitness) of each. 3. **Select** the n best. 4. **Clone** them; better antibodies get more clones. 5. **Hypermutate** clones; better antibodies mutate less. 6. Re-evaluate; keep the best for next generation; replace worst with random new antibodies (diversification). 7. Repeat until convergence.

6.2.3 CSA vs Genetic Algorithms

	CSA	GA
Inspiration	Immune system	Natural selection
Recombination	None (asexual)	Crossover
Variation	Hypermutation only	Mutation + crossover
Selection	Affinity-proportional cloning	Tournament / roulette
Strength	Maintains diversity, multimodal optimisation	Faster on unimodal problems

6.2.4 Affinity

A problem-specific score. For minimisation, affinity is often defined as `1 / (1 + f(x))` so higher affinity = better.

6.2.5 Applications

Pattern recognition, optimisation, anomaly detection, network security, multimodal function optimisation, robotics.

6.3 Why Python?

- Pure algorithm pedagogy — no networking, no GUI.
- `random` and list comprehensions express CSA in ~30 lines.
- Real applications would use NumPy/SciPy or specialised libraries (PyAIS, AISpy).

6.4 Code walkthrough

```
population = [random.randint(target - 50, target + 50) for _ in range(pop_size)]
```

Random initial antibodies near the target (just to keep numbers reasonable for a demo).

```
population.sort(key=lambda x: affinity(x, target))
best = population[:n_select]
```

Affinity = distance to target. Sort ascending so best (smallest distance) is first.

```
clones = [b + random.randint(-mutation_range, mutation_range) for b in best]
```

Clone & mutate. In a more faithful CSA, the *number* of clones and *amount* of mutation would depend on the antibody's affinity rank.

```
population = (best + clones)
population.sort(key=lambda x: affinity(x, target))
population = population[:pop_size]
```

Combine, sort, trim back to original size — the elitist replacement.

6.5 Likely viva questions

Q: What is the Clonal Selection Algorithm? A: An optimisation heuristic inspired by how B-cells in the immune system select, clone, and mutate to produce antibodies with high affinity to antigens.

Q: What is affinity? A: A measure of how well an antibody (candidate solution) matches the antigen (target / fitness goal). Higher affinity = better.

Q: How is CSA different from a Genetic Algorithm? A: CSA uses cloning + hypermutation only (no crossover). It maintains better diversity by also injecting random new antibodies each generation, making it good for multimodal problems where GAs may converge prematurely.

Q: What is hypermutation? A: A higher-than-normal mutation rate applied to clones to explore the neighbourhood of high-affinity solutions.

Q: Where is CSA used in practice? A: Anomaly/intrusion detection, function optimisation, pattern recognition, scheduling.

Q: Convergence criterion? A: Either a fixed number of generations or “no improvement for N generations”.

Practical 7 — AIS Pattern Recognition for Damage Classification

7.1 Aim

Apply **Artificial Immune System** pattern recognition to classify structural sensor readings as damaged or normal.

7.2 Theory

7.2.1 Artificial Immune Systems (AIS)

Computational models inspired by the vertebrate immune system, designed for: - Pattern recognition - Anomaly detection - Optimisation - Classification - Learning

7.2.2 Key AIS algorithms

- **Negative Selection Algorithm (NSA)** — generate detectors that *do not* match self; they will then match non-self (anomalies). Inspired by T-cell maturation in the thymus.
- **Clonal Selection (CLONALG)** — Practical 6.
- **Immune Network Theory** — antibodies stimulate/suppress each other.
- **Danger Theory** — discriminate based on “danger signals” rather than self/non-self.
- **Dendritic Cell Algorithms** — newer AIS approach.

7.2.3 AIS terminology mapped to ML

- **Antigen** = data sample (input pattern).
- **Antibody** = detector (model parameter / neuron analogue).
- **Affinity** = similarity (1/distance).
- **Self** = normal data; **non-self** = anomaly / damage.
- **Memory cells** = trained classifier parameters.

7.2.4 Affinity / similarity measures

- **Euclidean distance** (real-valued data)
- **Hamming distance** (binary strings)
- **Manhattan / city-block** distance
- **Cosine similarity**
- **Mahalanobis distance** (correlated features)

7.2.5 Structural Health Monitoring (SHM)

The motivating application: aircraft wings, bridges, and buildings have sensors (accelerometers, strain gauges). AIS classifiers process readings to flag damaged structures **before** catastrophic failure.

7.2.6 Why immune-inspired vs neural networks?

- AIS naturally handles **anomaly detection** (one-class learning) — useful when damaged samples are rare.
- Easier to interpret than deep nets.
- Continuous learning capability — can add new training examples online.
- Slower to converge than well-tuned NNs on large datasets.

7.3 Why Python?

Same reasoning as Practical 6: pedagogy, brevity, ecosystem (numpy, scipy, scikit-learn, AIS libraries available).

7.4 Code walkthrough

```
def train(samples, n_antibodies_per_class=5, mutation_range=0.05):
    normals  = [r for r, lab in samples if lab == 0]
    damaged  = [r for r, lab in samples if lab == 1]
    def grow(seed_pool):
        return [random.choice(seed_pool) + random.uniform(-mutation_range,
        mutation_range)
                for _ in range(n_antibodies_per_class)]
    return grow(normals), grow(damaged)
```

Training builds two antibody pools (one per class) by sampling labelled examples and slightly mutating them. This is a tiny clonal-selection-style training step.

```
def classify(reading, normal_pool, damaged_pool):
    def best_distance(pool):
        return min(abs(reading - ab) for ab in pool) if pool else float("inf")
    return "NORMAL" if best_distance(normal_pool) < best_distance(damaged_pool) else
        "DAMAGED"
```

Nearest-antibody classifier — the new reading is assigned to whichever class has the closest matching antibody. Conceptually similar to **k-nearest-neighbours (k=1)**.

7.5 Likely viva questions

Q: What is an Artificial Immune System? A: A family of bio-inspired algorithms based on the immune system, used for pattern recognition, classification, optimisation, and anomaly detection.

Q: What is the Negative Selection Algorithm? A: An AIS technique that produces detectors which do **not** match training (self) data; the detectors then identify anomalies

(non-self) when deployed.

Q: What is affinity in AIS? A: A similarity measure between an antigen (input) and an antibody (detector). High affinity ⇒ strong recognition.

Q: Why use AIS for damage classification? A: Damage data is often rare; immune-inspired approaches are good at one-class anomaly detection, can learn online, and have biological plausibility for adaptive systems.

Q: How would you compute affinity for binary strings? A: Hamming distance (count of differing bits), then convert to similarity by `1 – d / length`.

Q: How is this different from k-NN? A: k-NN stores all training points and finds the closest at classification time. AIS pattern recognition typically produces a *compressed* set of antibodies via cloning/mutation, sometimes with internal network dynamics, and may classify by affinity threshold rather than majority vote.

Practical 8 — Distributed Evolutionary Algorithm (DEAP)

8.1 Aim

Implement an **evolutionary algorithm** that evolves a population of candidate solutions toward an optimum across generations.

8.2 Theory

8.2.1 Evolutionary Computation umbrella

- **Genetic Algorithms (GA)** — bit/string chromosomes, crossover dominant.
- **Evolution Strategies (ES)** — real-valued vectors, mutation dominant.
- **Genetic Programming (GP)** — evolve programs (trees of code).
- **Evolutionary Programming (EP)** — historically focused on finite-state machines.
- **Differential Evolution (DE)** — vector difference mutation.

8.2.2 Generic GA pseudocode

```
Initialize random population
Evaluate fitness
While not done:
    Select parents (tournament/roulette)
    Crossover -> children
    Mutate children
    Evaluate children
    Form next generation (elitism + children)
Return best
```

8.2.3 Selection methods

- **Tournament** — pick k individuals at random; the best wins. Used in our code.
- **Roulette wheel** — selection probability ∝ fitness.
- **Rank** — selection probability ∝ rank; reduces “super-individual” dominance.

8.2.4 Crossover

Combine genetic material from two parents: - **Single-point** — split chromosomes at one point. - **Two-point** — two split points. - **Uniform** — each gene chosen from one parent at random. - For real numbers: arithmetic crossover (average), simulated binary crossover (SBX).

8.2.5 Mutation

Random change to maintain diversity. For integers/reals: add Gaussian or uniform noise. For bit strings: flip bit with probability p_m.

8.2.6 Elitism

Carry the best k individuals into the next generation unchanged. Prevents loss of the best solution to bad luck.

8.2.7 Fitness function

The objective we want to optimise. Must be cheap to evaluate (it's called population_size × generations times).

8.2.8 Convergence and premature convergence

- **Convergence** — population narrows around the optimum.
- **Premature convergence** — population gets stuck on a local optimum because diversity collapsed too fast. Combatted by: higher mutation, niching, larger population, restart strategies.

8.2.9 DEAP framework

Distributed Evolutionary Algorithms in Python. Provides: - `creator` — define custom fitness/individual classes. - `base.Toolbox` — register operators (select, mate, mutate, evaluate). - Algorithm helpers (`eaSimple`, `eaMuPlusLambda`). - Statistics, hall-of-fame.

We do not require DEAP to *understand* GA, but DEAP makes large-scale and distributed (multi-process / multi-machine) GA implementations easy.

8.2.10 Why "distributed" evolutionary algorithm?

GAs are **embarrassingly parallel**: fitness of each individual can be evaluated independently. Distribute the population across machines/cores → linear speedup. DEAP supports `multiprocessing.Pool` and SCOOP for this.

8.3 Why Python (and DEAP)?

- DEAP is the de-facto Python EC library and is **pure Python**.
- Python's high-order functions make it natural to express EAs (operators are first-class objects).
- Performance can be a concern, but for textbook problems it is fine.
- Java has Watchmaker, ECJ; C++ has GAlib; but Python's ecosystem (NumPy, scikit-learn integration) is unmatched for prototyping.

8.4 Code walkthrough

```
def evolve(domain, pop_size, generations, mutation_rate, mutation_range,
           elite_size, fitness_fn, mode="min", verbose=True):
```

A flexible evolutionary loop driven by user parameters.

```
new_pop = population[:elite_size]              # elitism
while len(new_pop) < pop_size:
    parents = random.sample(population, 3)      # tournament size 3
    parent  = sort_pop(parents)[0]
    other   = random.choice(population)
    child   = (parent + other) // 2             # arithmetic crossover (integer)
    if random.random() < mutation_rate:
        child += random.randint(-mutation_range, mutation_range)
        child = max(lo, min(hi, child))         # clamp to domain
    new_pop.append(child)
```

Each iteration: keep elites, then build offspring via tournament selection + arithmetic crossover + occasional mutation.

8.5 Likely viva questions

Q: What is an evolutionary algorithm? A: A population-based optimisation technique inspired by biological evolution, using selection, crossover, and mutation operators across generations to improve candidate solutions.

Q: What is the role of crossover vs mutation? A: Crossover combines existing good traits (exploitation); mutation introduces novelty (exploration). Both are needed.

Q: What is elitism? A: Keeping the best individual(s) unchanged across generations to prevent regression.

Q: How do you avoid premature convergence? A: Higher mutation rate, larger population, niching (penalise crowding), random restarts.

Q: What is a fitness function? A: A function that scores how good a candidate solution is. Drives selection. Must be cheap because it’s evaluated thousands of times.

Q: Why “distributed”? A: Fitness evaluations are independent and can be parallelised across CPU cores or machines, scaling speedup almost linearly.

Q: Difference between GA and CSA (P6)? A: GA uses crossover + mutation, CSA uses cloning + hypermutation only. GA mimics sexual reproduction, CSA mimics asexual immune-cell proliferation.

Practical 9 — Distributed Hotel Booking via Java RMI

9.1 Aim

A distributed hotel booking system using Java RMI: client books/cancels rooms; server maintains state.

9.2 Theory

All RMI fundamentals from Practical 2 apply. Additional concerns specific to a stateful service:

9.2.1 Server-side state

Unlike Practical 2 (pure functions), the hotel server maintains a `Map<roomNumber, guestName>`. This state must: - **Persist between calls** — keep it as a server-instance field. - **Be thread-safe** — multiple clients may call concurrently.

9.2.2 Concurrency in RMI

Java RMI **dispatches each incoming call in its own thread by default**. Without synchronisation, two clients could both pass the "is room free?" check simultaneously and both book the same room (a classic **race condition**).

We solve this by marking every method `synchronized` on the server. The JVM ensures only one thread executes any synchronized method on the same object at a time — guaranteeing **atomic** book/cancel operations.

9.2.3 Locking trade-offs

`synchronized` is coarse-grained (whole-object lock). For higher concurrency you could use: - `java.util.concurrent.ConcurrentHashMap` for the bookings map. - `ReadWriteLock` to allow many concurrent reads. - Optimistic concurrency (CAS).

For a textbook hotel with 10 rooms, a coarse lock is perfect.

9.2.4 Idempotency

A booking operation is **not** idempotent — calling `bookRoom(5, "Alice")` twice yields a "already booked" error the second time, *not* the same effect as one call. Cancellation has the same property. RMI's at-most-once semantics matter here.

9.2.5 Single point of failure / persistence

If the server crashes, all bookings are lost (in-memory only). Real systems persist to a database (Postgres, MySQL) and run multiple server replicas behind a load balancer with sticky sessions or distributed transactions.

9.3 Why Java RMI?

- Already covered in P2: built-in, type-safe, object-oriented.
- For a *real* hotel booking service you'd build REST/gRPC microservices on top of a database — but this practical's purpose is to show **stateful distributed objects** which is RMI's natural model.

9.4 Code walkthrough

`Hotel.java` (interface)

6 methods: `listAvailable`, `listBookings`, `bookRoom`, `cancelRoom`, `roomStatus`, `totalRooms`. The contract everything else relies on.

`HotelServer.java`

```
private final Map<Integer, String> bookings = new LinkedHashMap<>();
```

`LinkedHashMap` preserves **insertion order** — bookings list is shown in the order they happened, which is more user-friendly than a `HashMap`'s arbitrary order.

```
@Override
public synchronized String bookRoom(int roomNo, String guest) {
    if (roomNo < 1 || roomNo > TOTAL_ROOMS) return "FAIL: room ...";
    if (bookings.containsKey(roomNo))       return "FAIL: already booked ...";
    bookings.put(roomNo, guest.trim());
    return "OK: ...";
}
```

- `synchronized` provides the atomic check-then-set required for booking.
- Validates inputs to prevent garbage state.
- Returns a **status string** (OK/FAIL prefix). A more elegant API would throw a custom `BookingException`.

`HotelClient.java`

A 6-option menu loop (view available, view bookings, book, cancel, status, exit). Each menu choice maps directly to one remote call. The client is otherwise stateless — the server is the system of record.

9.5 Likely viva questions

Q: How does the server maintain state? A: Through fields (instance variables) of the `HotelServer` object, which lives for the lifetime of the JVM.

Q: How is concurrency handled? A: RMI dispatches each incoming call in its own thread; we mark every server method `synchronized` so only one runs at a time, preventing race conditions like double-booking.

Q: What if the server crashes? A: All bookings are lost (in-memory only). A production system would persist to a database and replicate the server.

Q: Could we build this as a REST API instead? A: Yes — that's what real systems do. The trade-off: REST is language-neutral and stateless (each request carries auth/session tokens) and scales horizontally; RMI is Java-only but more natural for object-oriented stateful services in a homogeneous environment.

Q: Why use `LinkedHashMap` instead of `HashMap`? A: To preserve booking insertion order for stable, predictable client output.

Q: What is at-most-once semantics and why is it relevant here? A: Each remote call executes 0 or 1 times. Important for non-idempotent operations like booking — we don't want duplicate bookings due to retries.

Q: How could you scale to many hotels / millions of rooms? A: Replace in-memory map with a database. Run multiple stateless app servers behind a load balancer. Add a cache layer (Redis). Use distributed transactions if global guarantees are required.

Practical 10 — Ant Colony Optimization for TSP

10.1 Aim

Apply **Ant Colony Optimization** to the **Travelling Salesman Problem**.

10.2 Theory

10.2.1 Travelling Salesman Problem (TSP)

Given N cities and pairwise distances, find the shortest route visiting **every city exactly once** and returning to start. Tours are cyclic permutations of cities.

- **NP-hard**: exact solution time grows as O(N!) (or $O(N^2 \cdot 2^N)$ with Held-Karp dynamic programming).
- For N > ~20 we need heuristics: nearest neighbour, 2-opt, simulated annealing, GA, ACO.

10.2.2 Swarm intelligence

A class of techniques inspired by collective behaviour of decentralised, self-organised social insects/animals. Members: - **Ant Colony Optimization (ACO)** - **Particle Swarm Optimization (PSO)** - **Bee Algorithms** - **Firefly Algorithm**

Common theme: simple agents, local rules, indirect communication, emergent global intelligence.

10.2.3 Biological inspiration for ACO

Real ants find shortest paths between nest and food via **pheromone trails**: 1. Ants wander randomly, depositing pheromone as they go. 2. Returning ants reinforce the path. 3. Pheromone evaporates over time. 4. Shorter paths get reinforced faster (more round-trips per minute) → pheromone accumulates → other ants are biased toward the shorter path → positive feedback finds the optimum.

10.2.4 ACO algorithm (Ant System for TSP)

For each iteration: 1. Place each ant on a random starting city. 2. Each ant builds a tour by probabilistically choosing the next city using `P(j | current=i) ∝ τ(i,j)^α × η(i,j)^β` where: - τ(i,j) = pheromone on edge (i,j) - η(i,j) = heuristic = 1/distance(i,j) - α controls importance of pheromone, β of distance. 3. After all ants finish, **evaporate** pheromone: τ ←

(1−ρ)·τ. 4. **Deposit** pheromone: each ant adds Q/length on the edges of its tour. Shorter tours deposit more. 5. Track the best tour found so far.

10.2.5 ACO variants

- **Ant System (AS)** — original (this practical).
- **Elitist AS** — extra deposit on the best-so-far tour.
- **Max-Min Ant System (MMAS)** — pheromone bounds prevent stagnation.
- **Ant Colony System (ACS)** — additional local pheromone update during tour construction.
- **Rank-based AS**.

10.2.6 Convergence

Pheromone reinforcement creates positive feedback that biases ants toward good tours, while evaporation prevents premature lock-in. Tuning α, β, ρ, Q, ant count, and iterations is critical.

10.2.7 Applications beyond TSP

Vehicle routing, scheduling, network routing protocols (AntNet), bioinformatics, image processing.

10.3 Why Python?

- ACO is iterative with $O(N^2 \times \text{ants} \times \text{iterations})$ — feasible in Python for small TSP instances.
- Random sampling and probabilistic selection are natural to express with `random` and list comprehensions.
- For huge TSPs (thousands of cities), Cython, NumPy or C++ implementations are used — but pedagogically Python is unbeatable.

10.4 Code walkthrough

```
def choose_next_city(current, unvisited, pheromone, dist, alpha, beta):
    weights = []
    for j in unvisited:
        tau = pheromone[current][j] ** alpha
        eta = (1.0 / dist[current][j]) ** beta if dist[current][j] > 0 else 0
        weights.append(tau * eta)
    total = sum(weights)
    if total == 0: return random.choice(unvisited)
    r = random.random() * total
    acc = 0
    for j, w in zip(unvisited, weights):
        acc += w
        if acc >= r: return j
    return unvisited[-1]
```

Roulette-wheel selection based on `τ^α · η^β`. Probability of choosing city j is its weight divided by total weight.

```
for i in range(n):
    for j in range(n):
        pheromone[i][j] *= (1 - evap)            # evaporation

for tour, length in all_tours:
    deposit = q / length
    for i in range(len(tour) - 1):
        a, b = tour[i], tour[i+1]
        pheromone[a][b] += deposit              # reinforcement
        pheromone[b][a] += deposit
    pheromone[tour[-1]][tour[0]] += deposit
    pheromone[tour[0]][tour[-1]] += deposit
```

Two phases per iteration: evaporation (uniform decay) followed by deposit (each ant lays Q/length, so shorter tours deposit more on their edges).

10.5 Likely viva questions

Q: What is TSP and why is it hard? A: Find the shortest tour visiting every city once and returning. NP-hard — no polynomial-time exact algorithm is known; brute force is O(N!).

Q: How does ACO solve TSP? A: Many “ants” build tours by probabilistic city selection biased by pheromone trails and inverse distance. Pheromone is reinforced on shorter tours and evaporates over time, gradually concentrating on the best path.

Q: Role of α and β? A: α weights pheromone (memory); β weights distance heuristic (greed). High α/low β → ants follow herd; low α/high β → ants behave like nearest-neighbour heuristic. Typical values: α=1, β=2-5.

Q: Why pheromone evaporation? A: Without it, early random tours keep dominating and the search gets stuck (pheromone explosion). Evaporation lets the colony “forget” weak trails.

Q: ACO vs Genetic Algorithm for TSP? A: ACO is constructive (builds tours edge by edge); GA is generative (mutates/crosses whole tours). Both are heuristics; ACO often

performs slightly better on TSP-like routing problems.

Q: What is swarm intelligence? A: Collective problem-solving by decentralised agents following simple local rules (ants, bees, birds). No central controller — global behaviour is emergent.

Q: What is the time complexity of one ACO iteration? A: O(ants × N^2) — each ant visits all N cities, and each step considers N candidate next-cities.

Cross-Cutting Viva Questions

These often come at the start or end and test broad understanding.

Q: Differentiate between RPC and RMI. A: | | RPC | RMI | |–|—–|—–| | Paradigm | Procedural | Object-oriented | | Languages | Language-neutral | Java only | | Marshalling | XDR / protobuf / JSON | Java Object Serialization | | Pass-by | Value | Value (Serializable), Reference (Remote) |

Q: Differentiate between Hadoop and Spark. A: Hadoop MapReduce reads/writes intermediate results to HDFS (slow, fault-tolerant); Spark keeps data in memory (RDDs/DataFrames), making iterative algorithms 10–100× faster. Both can run on YARN.

Q: What does "distributed" mean in your practicals? A: Components running in separate processes (often on separate machines) and coordinating via message passing — TCP sockets in P1, P2, P9; HDFS+YARN in P3; conceptual simulation in P5.

Q: Difference between fuzzy logic and probability? A: Probability deals with the likelihood of binary events; fuzzy deals with degrees of belonging to vague categories. Different mathematics, different intent.

Q: Why are bio-inspired algorithms (CSA, GA, ACO, AIS) useful? A: Many real-world problems are NP-hard or non-differentiable; classical exact methods fail. Bio-inspired algorithms are general-purpose, derivative-free, robust to noise, and often find good (if not provably optimal) solutions in reasonable time.

Q: What is the difference between exploration and exploitation? A: **Exploration** — search the solution space widely (high mutation, randomness). **Exploitation** — refine known good areas (greedy selection, low mutation). Good optimisation needs both — start exploring, then exploit. The trade-off is fundamental to all heuristics in P6, P7, P8, P10.

Q: Why do we use pseudo-distributed Hadoop in the lab and not standalone? A: To exercise the **real** distributed pipeline: NameNode + DataNodes for HDFS storage, ResourceManager + NodeManagers for YARN scheduling. Standalone runs everything in one JVM — faster, but doesn't show the architecture.

Q: What are common failure modes in distributed systems? A: Network partition (split brain), node crash, message loss, message duplication, message reordering, byzantine faults (malicious or buggy nodes), clock skew. The CAP theorem says you can guarantee at most 2 of {Consistency, Availability, Partition tolerance}.

Q: What is the CAP theorem? A: In a distributed data store you can guarantee at most two of: **Consistency** (all nodes see the same data), **Availability** (every request gets a response), **Partition tolerance** (system keeps working despite network splits). You always must choose P; the real trade-off is C vs A.

Q: What is at-most-once vs exactly-once semantics? A: At-most-once = call executes 0 or 1 times (RMI default). Exactly-once is theoretically impossible in the presence of arbitrary failures but can be approximated with idempotency keys + retries + deduplication on the server.

Glossary

Term	Meaning
Affinity	Similarity score in immune/CSA algorithms
Antibody	A detector / candidate solution in AIS
Antigen	Input pattern / data sample in AIS
ApplicationMaster	Per-application coordinator in YARN
At-most-once	Call executes 0 or 1 times
Block (HDFS)	128 MB chunk of an HDFS file
Combiner	Map-side mini-reducer to compress shuffle data
Container (YARN)	Resource bundle (CPU+RAM) for a task
Convergence	Population narrowing around the optimum
Crispset	Classical Boolean set
Crossover	GA operator combining two parents
DataNode	HDFS worker storing blocks
Daemon	Background process listening for requests
Elitism	Keeping best individuals across GA generations
Evaporation (ACO)	Decay of pheromone over time
Fitness	Score driving selection in EAs
Fuzzy set	Set with [0,1] membership values
HDFS	Hadoop Distributed File System
Hypermutation	Higher-than-normal mutation rate on clones (CSA)
Marshalling	Serializing objects to bytes
Membership function	$\mu(x) \in [0,1]$
MapReduce	Two-phase parallel data processing model
NameNode	HDFS master holding metadata
NodeManager	YARN per-node agent
NP-hard	Class of problems with no known polynomial algorithm
Pheromone (ACO)	Reinforcement value on graph edges
Proxy / Stub	Client-side surrogate for a remote object
Pyro4	Python Remote Objects framework
Registry (RMI)	RMI naming service on port 1099
Remote (Java)	Marker interface for RMI-callable interfaces
RemoteException	Mandatory exception on RMI methods
ResourceManager	YARN cluster scheduler (port 8032)

Term	Meaning
RMI	Remote Method Invocation (Java)
Round Robin	Cyclic load-balancing algorithm
RPC	Remote Procedure Call
Selection (GA)	Choosing parents based on fitness
Serialization	Converting objects to bytes
Shuffle and Sort	MapReduce phase grouping values by key
Skeleton	Server-side counterpart to a stub
Stateless	Server keeps no per-client information
Sticky session	Client always routed to the same server
Swarm intelligence	Collective behaviour by simple agents
Synchronous	Caller blocks until result returns
TSP	Travelling Salesman Problem
UnicastRemoteObject	RMI superclass that exports an object
URI (Pyro4)	`PYRO:obj_xxx@host:port`
Writable	Hadoop's serialization interface
YARN	Yet Another Resource Negotiator

End of document. Good luck with the viva — read each practical's section once carefully, then re-skim the cross-cutting Q&A. Most examiners only have time for 4–6 questions, drawn predominantly from the "Likely viva questions" lists per practical.